CM2606 Data Engineering - Summary Report

Module Code & Name	CM2606 Data Engineering
Report Title	End-to-End Sales Analytics Pipeline: Olist Brazilian E-Commerce
Student Name	*Srinath Thilakarathne*
RGU ID	*2313080*
IIT Student ID	*20211530*

---

1. Introduction

This report documents the design and implementation of an end-to-end data pipeline built on the Olist Brazilian E-Commerce dataset. Raw transactional CSV files are ingested into a Parquet-based data lake (bronze layer), cleaned and standardised by a distributed PySpark job (silver layer), and finally loaded into a PostgreSQL star-schema data warehouse (gold layer). The complete workflow is orchestrated by a single Apache Airflow DAG, enabling the entire pipeline to run in one execution without manual intervention.

---

2. Dataset Selection

The dataset is the **Olist Brazilian E-Commerce** public dataset (Kaggle, CC-BY-NC-SA-4.0), covering approximately 100,000 orders placed on the Olist marketplace between 2016 and 2018. It comprises eight related CSV files spanning customers, orders, order items, payments, reviews, products, sellers, and a Portuguese-to-English product category translation table. The dataset was used in full; no subset was required as PySpark handled the volume without resource issues.

---

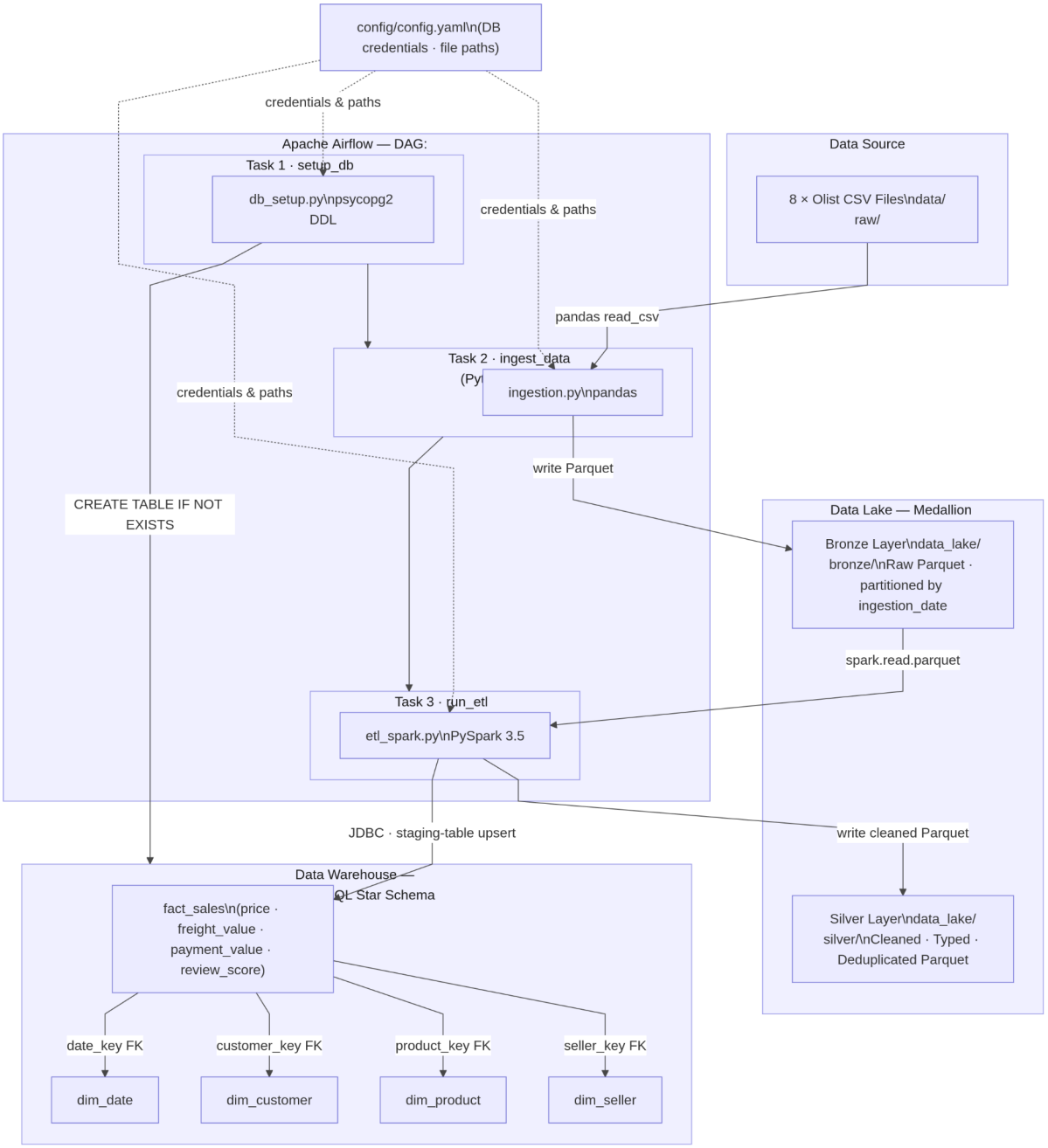
3. Insight Generation Mechanism

The chosen insight generation mechanism is a **Sales Performance BI Dashboard** to be connected directly to the PostgreSQL data warehouse. The dashboard would specifically enable analysts to answer:

- **Revenue trend analysis** - monthly and quarterly revenue (BRL) over the 2016–2018 period to identify seasonal patterns and growth trajectory.
- **Geographic sales distribution** - revenue and order volume broken down by Brazilian state and city, identifying high- and low-performing regions.
- **Product category performance** - top-selling categories by items sold and revenue, enabling category managers to prioritise stock and promotions.
- **Customer review quality** - average review score per category and seller, surfacing service quality issues.

The star schema is directly designed to serve these queries: `fact_sales` holds the measurable facts (price, freight, payment, review score) and the four dimension tables (date, customer, product, seller) provide the slicing axes. A BI tool such as Apache Superset or Metabase would connect to PostgreSQL via a read-only user and surface these views as interactive charts.

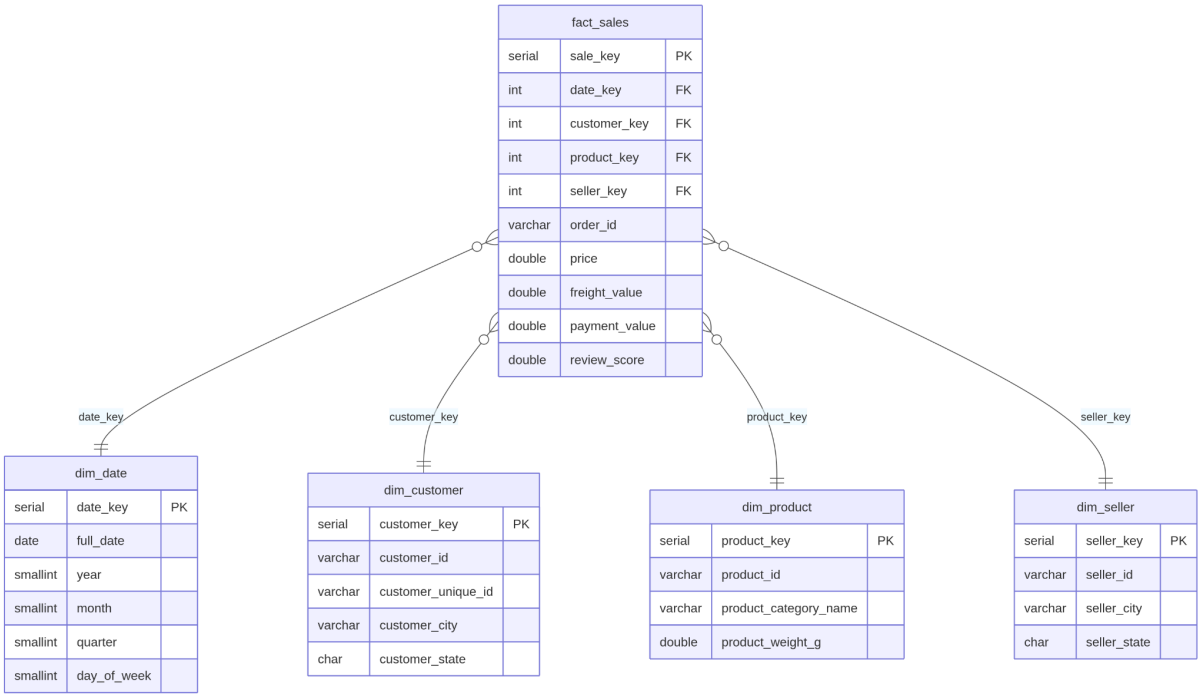
4. Pipeline Design

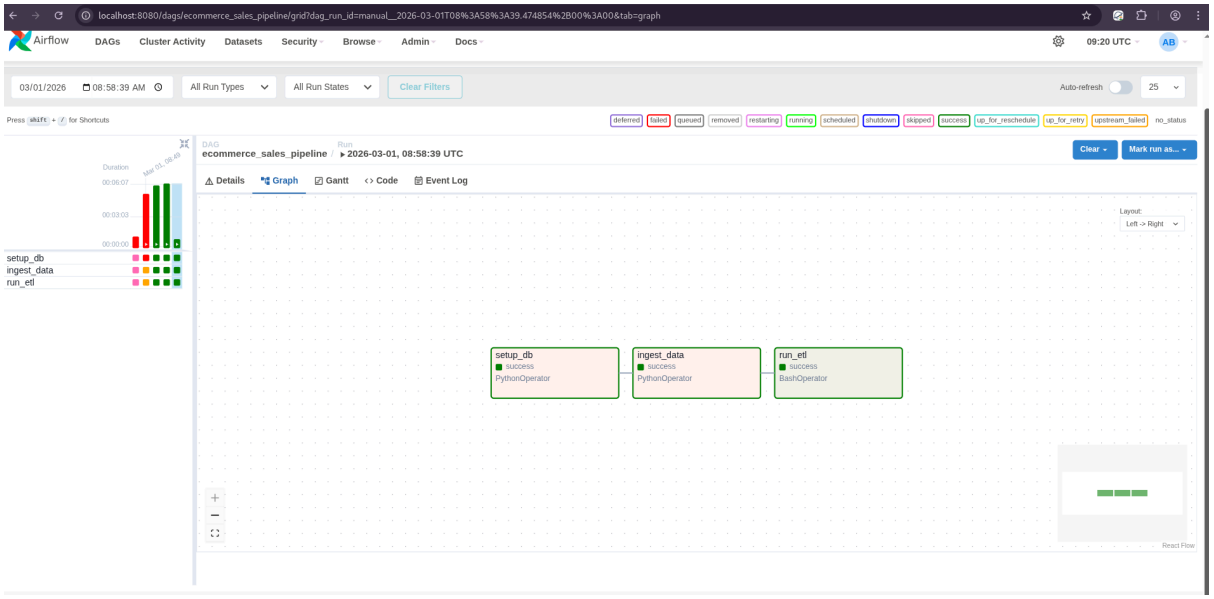


Components:

- **Data Source** - Eight CSV files downloaded to `data/raw/` on the local machine.
- **Ingestion Flow** (`ingestion.py`) - pandas reads each CSV and writes it as Parquet to `data_lake/bronze/`, partitioned by `ingestion_date` to support future incremental loads and full audit traceability.

- **Data Lake** - Local filesystem organised into `bronze/` (raw, immutable) and `silver/` (cleaned) layers following the medallion architecture pattern.
- **ETL Flow** (`etl_spark.py`) - A PySpark job reads from bronze, applies four transformations (see Section 5), writes clean data to silver, then loads the PostgreSQL star schema using a staging-table upsert pattern for idempotency.
- **Data Storage** - PostgreSQL star schema with one fact table (`fact_sales`) and four dimension tables (`dim_date`, `dim_customer`, `dim_product`, `dim_seller`). Surrogate keys are managed via `SERIAL` columns; foreign key indexes are created on the fact table for join performance.
- **Orchestration** (`ecommerce_pipeline_dag.py`) - An Airflow DAG (`ecommerce_sales_pipeline`, schedule `@once`) chains three tasks sequentially: `setup_db → ingest_data → run_etl`.





---

5. Technical Implementation

Tool Comparison

Component	Option Considered	Option Selected	Justification
Ingestion	PySpark, pandas	**pandas**	CSVs are read once per run; pandas is simpler and sufficient for single-node ingestion without requiring a Spark context
Distributed Processing	Apache Flink, Dask, PySpark	**PySpark 3.5**	Industry-standard for batch ETL; native Parquet support; DataFrame API familiar from lecture content
Data Lake Format	CSV, Avro, Parquet	**Parquet**	Columnar storage yields faster analytical reads and significant compression versus raw CSV
Data Warehouse	MySQL, MongoDB, PostgreSQL	**PostgreSQL**	Mature ACID-compliant RDBMS; strong JDBC support for PySpark; free and open-source

Orchestration	Prefect, Luigi, Airflow	**Apache Airflow 2.8**	De-facto industry standard; DAG-based dependency management; built-in logging and retry logic

ETL Transformations

Four cleaning steps are implemented in the silver layer:

1. **Deduplication** - `dropDuplicates()` scoped to natural keys (e.g., `order_id`) across all DataFrames.
2. **Missing Value Handling** - `review_score` nulls filled with the median (via `approxQuantile`) to avoid biasing averages; rows missing `order_id` or `customer_id` dropped as they cannot participate in any join.
3. **Data Type Conversion** - Timestamp strings parsed with `to_timestamp()` cast to `TimestampType`; `price`, `freight_value`, and `payment_value` cast to `DoubleType` for correct aggregation.
4. **Data Standardisation** - City and state fields uppercased to eliminate case inconsistencies; product category names translated from Portuguese to English via a left join against the translation lookup table.

Additional Quality Features

- Ingestion logging records row counts and output file sizes per dataset.
- Upsert logic (`INSERT … ON CONFLICT DO NOTHING`) ensures the pipeline is idempotent and safe to re-run.
- Config-driven design (`config.yaml`) keeps all environment-specific settings (DB credentials, paths) outside the codebase.

---

6. Discussion and Conclusion

Challenges: The main implementation challenge was loading dimension surrogate keys into the fact table without Spark's native upsert support. This was resolved using a staging-table pattern: PySpark writes the raw fact data to a temporary PostgreSQL table via JDBC, then a psycopg2 connection performs the surrogate key resolution and final insert using a single SQL statement, keeping the logic in the database where joins are most efficient.

*